Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 12 - Part 1

Constant Correctness and Static Functions

THIS WEEK'S AGENDA

01. Parameters with const keyword
02. Interesting cases with const
03. Static Variables - Review
04. Static Functions
05. Factory Design Pattern

Parameters With CONST Keyword

CONST PARAMETER WITH PRIMITIVE TYPES

- For basic/primitive data types like integer or double, marking a parameter as ***const*** does not offer many advantages because these parameters are passed by value.
- This means that a copy of these parameters is made which is then passed to the function.

```
void displayDistance(const int feet, const double inches) {
    // Cannot modify a constant integer parameter
    feet = 10;
    // Cannot modify a constant double parameter
    inches = 2.6;
    std::cout << "Feet: " << feet << " inches: " << inches << std::endl;
}
```

CONST PARAMETER WITH REFERENCE TYPES

- For larger objects, passing a parameter by reference and with ***const*** is more efficient than passing it by value.
- This prevents a copy of the object being made, while ***const*** ensures that the object won't be modified.
- Using ***const*** here saves the cost of copying a potentially large object while also ensuring that the function can not alter the message.

```
void displayDistance(const std::string &message) {
    // Error: message is constant
    message += " - additional text here.";
    std::cout << message << std::endl;
}
```

CONSTANT POINTER PARAMETERS

- When using pointers, marking a parameter as constant is valuable because it clarifies whether the function will modify the data being pointed to or not.
- **Pointer to Constant:** The data pointed to cannot be modified, but the pointer itself can be reassigned.

```
void processArray(const double* arr) {
    // Cannot modify the value pointed to
    *arr = 9.2;
    /* It is ok to change the location
    being pointed to by the pointer */
    arr = nullptr;
}
```

CONSTANT POINTER PARAMETERS

- **Constant Pointer to a Constant:** In this case, neither the pointer nor the data it points to can be modified.

```
void processArray(const double* const arr) {
    // Cannot modify the value pointed to
    *arr = 9.2;
    // Cannot modify the pointer itself either
    arr = nullptr;
}
```

CONSTANT PARAMS IN FUNCTION DECLARATIONS

- In function declarations (prototypes), marking parameters as const helps to clarify intentions of the parameter not being modifiable both to the programmer and the compiler.
- For example:

```
double* computeSpeeds(const int count, const double
*distance, const double *time);
```

Intersting Cases With Const

INTERESTING CASES WITH CONST

- Let's say we have the following code:

```cpp
void stringOne(std::string &s) {
    s += "Hello";
    std::cout << s << std::endl;
}

void stringTwo(const std::string &s) {
    /* String s is const here but passing it to
    the stringOne function that allows a non-const
    parameter of string type means that it would
    expect it to be modifiable there which is not
    allowed by stringTwo */
    stringOne(s);
    std::string localCopy = s;
    // This is okay since localCopy is non-const
    stringOne(localCopy);
}
```

INTERESTING CASES WITH CONST

- Assume that “X” is any data type and think of the answers to the following questions:
  - What does “const X* p” mean?
  - What’s the difference between “const X* p”, “X* const p”, and “const X* const p”?
  - What does “const X& x” mean?
  - What do “X const& x” and “X const* p” mean?
  - Does “X& const x” make any sense?
- You may find answers to these questions at: Const Correctness - ISOCPP

Static Variables - Review

STATIC VARIABLES – REVIEW

- Recall that we have previously studied static variables in the course.
- A **static variable** (also called a **class variable**) maintains only one copy of itself for the entire class.
- Which means that each object of that class does not have a separate copy of the static variable.
- Instead all the objects of that class share the same copy of the static variable.

Static Functions

STATIC FUNCTIONS

- In C++, a static function is a function that belongs to a class but does not operate on a specific instance of that class.
- Static functions are declared with the **static** keyword and have unique properties and use cases.
- Static functions have two interesting properties:
  - They belong to the class itself, not to any particular object of it.
  - They can be called directly on the class without creating an instance of the class.
- Let's look at an example:

STATIC FUNCTIONS

```cpp
class Calculator {
    public:
        static int subtract(int a, int b) {
            return a - b;
        }
};

int main() {
    int result = Calculator::subtract(5, 6);
    std::cout << "Subtraction result: " << result << std::endl;
}
```

ACCESS RESTRICTIONS OF STATIC FUNCTIONS

- Static functions can only access other static members (variables/functions) of the class.
- They do not have access to the “this” keyword, as they do not belong to any specific instance.
- Let’s again look at an example:

ACCESS RESTRICTIONS OF STATIC FUNCTIONS

```cpp
class Counter {
    private:
        static int count;
        std::string counterName;
    public:
        static void increment() {
            // Can access the static count variable
            count++;
            // Cannot access non-static members
            counterName = "Counter One";
        }
};

// Initialize static member outside class
int Counter::count = 0;
```

STATIC FUNCTIONS CANNOT BE VIRTUAL

- Static functions cannot be virtual, which means that they do not support polymorphism or dynamic dispatch.
- This is because virtual functions work with instances of a class as we have seen in the past.
- Conversely, static functions are linked with the class itself and not with a particular instance of the class.

USE CASES OF STATIC FUNCTIONS

- Static functions are often used for utility functions that do not particularly need an object of a class to work.
- Some common examples include: mathematical calculations and logging functions.

```cpp
class Logger {
    public:
        static void log(const std::string& message) {
            std::cout << "Log: " << message << std::endl;
        }
};

int main() {
    Logger::log("This is a message");
}
```

STATIC FUNCTIONS

- Static functions are useful when a function's logic is specific to the class but doesn't need to interact with the instance's state.
- This can help encapsulate logic within the class itself, rather than defining a function globally.
- The lifetime of static functions is independent of any of the class instances.
- Static functions will exist as long as the program is running, and they also do not require any instance initialization to be accessible.

Factory Design Pattern

FACTORY DESIGN PATTERN

- A very common use of static functions in in implementing the Factory Design pattern.
- In this design pattern a static function is used to create an instance of the class without us having to explicitly create an instance of that class using a constructor (default or user defined) in the main function.
- Let’s look at an example of this:

FACTORY DESIGN PATTERN

```cpp
class Item {
    public:
        static Item createDefaultItem() {
            // Creates a default Item instance
            return Item();
        }
};

int main() {
    /* Create an Item instance using static factory
    method */
    Item defaultItem = Item::createDefaultItem();
}
```

Summary

SUMMARY

- Const correctness is an important aspect of type correctness in C++.
- There are several variations of const correctness, each with its own specifics.
- Static functions, just like static variables, are linked with the class rather than an instance of it.
- Static functions can be used to construct an instance of the class in the way shown in the Factory Design pattern.

READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 11

ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science